

3D 섯다 최적 베팅 전략 검증 & 버그·패치 분석 보고서

작성일: 2026-09-13 대상 파일: 3D_섯다.html (오류_수정본) 방법론: 원본 게임 로직 Node.js 이식 후 반복 시뮬레이션 총 시뮬레이션 규모: 약 2억 7,000만 판

목차

- 1. 개요 및 방법론
- 2. 시뮬레이션 결과
 - 2.1 손패 티어별 기초 통계
 - 2.2 베팅 전략 비교
 - 2.3 기본판돈 비율(f) 최적화
 - 2.4 최종 통합전략 장기 검증
- 3. 최종 권고 전략
- 4. 버그 및 문제점 상세 분석
- 5. 패치 권장안
- 6. 한계 및 유의사항

1. 개요 및 방법론

본 보고서는 3D 섯다(3D_섯다.html)의 카드 배분 로직(SeotdaEngine.evalHand), AI 의사결정 로직(Game.processAiTurn), 베팅 처리 로직(Game.handlePlayerBet)을 Node.js로 100% 동일하게 이식하여, 브라우저에서 직접 플레이하는 대신 대규모 반복 시뮬레이션으로 ① 가장 수익률이 높은 손패별 베팅 전략과 ② 기본판돈을 잔고 대비 얼마로 설정해야 하는지를 검증했다. 검증은 4단계로 진행했으며, 각 단계의 표본 규모는 이전 소규모 검증(수십만~수백만 판) 대비 최대 100배까지 늘렸다.

파트	검증 내용	표본 규모
A	손패 티어별 등장확률·승률	2,000만 판
B	베팅 전략 5종 비교	전략당 1,000만 판 (총 5,000만 판)
C	기본판돈 비율(f) 17개 구간 정밀 스윙	구간당 1,000만 판 (총 1억 7,000만 판)
D	최종 통합전략 장기(1,000판) 검증	3,000만 판 (1,000판×3만 세션)

2. 시뮬레이션 결과

2.1 손패 티어별 기초 통계 (2,000만 표본)

티어	족보	등장확률	승률	무승부	패배
0	광뎡	1.580%	100.00%	0.00%	0.00%
1	뎡	5.263%	95.48%	0.00%	4.52%
2	중간족보(알리~세륙)	12.623%	88.21%	0.65%	11.13%
3	7~9끗	28.946%	60.71%	10.14%	29.15%
4	4~6끗	22.627%	36.25%	7.68%	56.07%
5	0~3끗/망통	28.961%	10.33%	7.58%	82.09%

※ 광뎡은 카드 구성상 광뎡끼리 충돌이 수학적으로 불가능하여 2,000만 표본에서 단 한 번도 지거나 비기지 않음(필연적 100% 승률).

2.2 베팅 전략 비교 (전략당 1,000만 판, 잔고 스냅샷 고정 EV)

전략	판당 EV	기본판돈 대비	표준편차
다이 없이 항상 콜 (비교군)	-157.9원	-15.79%	2,535원
약패만 다이, 레이즈 없음	+81.3원	+8.13%	2,203원
추천 전략 (광뎡=올인·뎡=하프·중간족보=하프·7~9끗=콜·나머지=다이)	+1,282.1원	+128.21%	17,986원
공격형 (뎡도 무조건 올인 추격)	+4,925.9원	+492.59%	36,170원
초타이트 (뎡 이상만 참여)	+454.4원	+45.44%	18,047원

2.3 기본판돈 비율(f) 정밀 스윙 (구간당 1,000만 판)

f (기본판돈/잔고)	파산율(500판 내)	판당 기하성장률	500판 후 중앙값
0.3%	0.00%	+0.6873%/판 (최고)	2,464,194원
0.5%	0.00%	+0.6862%/판	2,484,615원
0.7%	0.00%	+0.6804%/판	2,451,681원
1.0%	0.00%	+0.6796%/판	2,481,250원
1.2%	0.00%	+0.6767%/판	2,385,637원
1.5%	0.00%	+0.6662%/판	2,309,747원
2.0%	0.00%	+0.6451%/판	2,113,428원
3.0%	0.01%	+0.5656%/판	1,479,264원
4.0%	0.19%	+0.4634%/판	908,518원
5.0%	1.18%	+0.3208%/판	460,472원
7.0%	12.38%	-0.1214%/판 (적자 전환)	64,853원
8.0%	30.14%	-0.3573%/판	13,364원
10.0%	58.24%	-0.8428%/판	70원
12.0%	75.46%	-1.3741%/판	39원
15.0%+	93%+	-2.7%~-6.1%/판	사실상 0원

0.3%~1.5% 구간은 성장률이 사실상 평평한 고원을 이루며(0.666%~0.687%, 차이 무의미), **7% 지점부터 기대수익이 적자로 전환**, 10% 이상은 500판 내 파산율이 50%를 넘는 명백한 필패 구간이다.

2.4 최종 통합전략 장기 검증 (f=1%, 1,000판×3만 세션 = 3,000만 판)

구간	1,000판 후 잔고
하위 5%	1,328,810원
하위 25%	12,473,778원
중앙값	74,811,015원
상위 75%	512,069,473원
상위 95%	12,226,594,223원
파산율	0.00% (3만 세션 전원 생존)

시작 잔고 100,000원 대비 1,000판 후 중앙값 기준 약 748배 성장. 3만 회 독립 시행에서 단 한 번도 파산하지 않았다.

3. 최종 권고 전략

내 패	최초 액션	재레이즈 대응
광땡	올인	무조건 콜/올인 추격
땡	하프	요구액 ≤ 기본판돈×6이면 콜, 초과시 다이
중간족보	하프	요구액 ≤ 기본판돈×3이면 콜, 초과시 다이
7~9끗	콜	요구액 ≤ 기본판돈×1이면 콜, 초과시 다이
4~6끗 / 0~3끗·망통	다이	-

기본판돈 규칙: 항상 현재 잔고의 0.5%~1% 사이로 설정(수학적 정점은 0.3~0.5%이나 1%까지 차이가 통계적으로 무의미하므로 계산 편의상 "잔고의 1%" 권장). 잔고가 ±20~30% 변할 때마다 재조정.

4. 버그 및 문제점 상세 분석

시뮬레이션 과정에서 게임 코드(3D_셋다.html) 자체의 구조적 문제 6가지를 발견했다. 심각도 순으로 정리한다.

CRITICAL 4.1 AI 잔고가 전혀 검증되지 않아 판돈이 무한정 불어남

문제: 코드 전체를 검색해도 AI 측 잔고를 나타내는 변수(aiBalance , aiMoney 등)가 존재하지 않는다. processAiTurn() 내부에서 AI가 콜/레이즈할 때 (998~1002번째 줄, 1012~1013번째 줄) this.potMoney += aiTotal 형태로 잔고 검증 없이 바로 팻에 돈을 더한다.

```
const aiTotal = playerBet + extra;
this.potMoney += aiTotal; // ← AI 잔고 체크 없음
Renderer3D.addChips(aiTotal);
this.aiRaiseExtra = extra;
```

결과적으로 플레이어가 강패(땡 이상)를 들고 재레이즈를 반복하면, AI는 한도 없이 계속 "따라오거나 더 없기" 때문에 판돈이 기하급수적으로 불어난다. 실제 시뮬레이션에서 300판 이내에 판돈이 수억~수천억 원까지 치솟는 사례가 다수 발생했으며, 이는 본 보고서 2.4절의 "1,000판 후 중앙값 7,481만원" 같은 극단적 수치의 핵심 원인이다.

DESIGN 4.2 다이(기권) 시 판돈이 누구에게도 가지 않고 소멸

문제: 905~910번째 줄, 플레이어가 다이를 선택하면 potMoney (이미 넣은 엔티와 베팅액 포함)는 그대로 둔 채 startNewRound() 가 호출되고, 그 안에서 this.potMoney = 0 으로 초기화된다(875번째 줄). 즉 그 돈은 AI에게 가지도, 하우스 수익으로 집계되지도 않고 그냥 사라진다. 플레이어가 약용할 수 있는 부분은 아니지만, 게임 내 총 통화량이 보존되지 않는 회계 불일치이며, 나중에 "하우스 레이크"나 "통계(누적 승패 금액)" 기능을 넣을 때 반드시 손봐야 한다.

DESIGN 4.3 최초 베팅(콜/하프/쿼터)으로는 AI를 절대 폴드시킬 수 없음

문제: 977번째 줄의 폴드 조건은 playerBet >= this.roundBaseBet * 2 다. 그런데 라운드 시작 시 팻은 항상 roundBaseBet * 2 이므로(880번째 줄), 최초 액션에서 콜은 정확히 roundBaseBet * 1, 하프는 roundBaseBet * 1, 쿼터는 roundBaseBet * 0.5 가 되어 셋 다 폴드 임계값(2배)에 절대 도달하지 못한다. 오직 올인이나, 레이즈 공방이 벌어져 팻이 커진 뒤의 재레이즈만 AI를 접게 만들 수 있다. 결과적으로 "하프/쿼터" 버튼은 최초 액션에서는 사실상 심리적 블러핑 기능을 전혀 하지 못하는 죽은 옵션이 되어버렸다.

LOGIC 4.4 foldBias의 중간 구간(0.26)이 실제로는 도달 불가능한 데드 코드

문제: 971번째 줄에서 normalizedStrength < 0.08 → 0.52 , < 0.18 → 0.26 , 그 외 → 0.06 으로 3단계 폴드 확률을 설계했지만, normalizedStrength = (score-100)/900 이고 실제 evalHand() 가 반환하는 점수는 100~110(꽃), 700~750(중간족보), 801~810(땡), 990~1000(광땡) 뿐이다. 0.08~0.18 구간은 점수 172~262에 해당하는데 이 범위에 속하는 족보가 게임 내에 하나도 없다. 즉 의도했던 "약함/중간/강함" 3단계 폴드 반응이 실제로는 "꽃(80.5%, 52% 폴드확률)"과 "그 이상(19.5%, 6% 폴드확률)"의 2단계로만 작동하고 있다. AI의 다이 반응이 설계 의도보다 훨씬 극단적으로 이분화되어 있다.

STABILITY 4.5 재레이즈 공방에 최대 횟수 제한이 없음

문제: 937번째 줄에서 플레이어가 재레이즈하면 processAiTurn(actualExtra, true) 가 다시 호출되고, AI가 또 shouldRaise 로 판정되면(강패일수록 최대 72%) 995~1003번째 줄에서 다시 재레이즈가 발생한다. 이 재귀 호출 횟수를 제한하는 코드(예: reraiseCount , maxReraise)가 어디에도 없다. 확률상 무한히 이어지진 않지만, 몇 차례만 반복돼도 팻이 기하급수적으로 커지며(4.1의 버그와 결합), 극단적인 경우 potMoney 가 JS의 안전한 정수 범위(2^53)에 가까워지거나 UI 표시가 비정상적으로 길어지는 등 안정성 문제로 이어질 수 있다.

MINOR 4.6 정수 내림(Math.floor) 처리로 인한 사소한 손실

문제: 무승부 시(1042번째 줄) Math.floor(this.potMoney / 2) 만 플레이어에게 돌아가므로, 팻이 홀수 금액일 때 1원이 시스템에서 소멸한다. 또한 947~948번째 줄에서 팻이 매우 작을 때(1~3원) 하프/쿼터 베팅액이 0으로 계산되어 951번째 줄의 예외 처리(bet = this.roundBaseBet)가 개입, 사용자가 "하프"를 눌렀는데 실제로는 "콜"과 동일한 금액이 베팅되는 등 사소한 UX 혼선이 있다.

5. 패치 권장안

5.1 [4.1 대응] AI 잔고 상한 도입 (최우선)

라운드 시작 시 this.aiBalance 를 초기화하고(예: 플레이어 잔고와 동일하거나, 별도 정책으로 roundBaseBet 의 N배로 고정), AI가 콜/레이즈할 때마다 이 값을 차감하도록 수정한다.

```
// 라운드 시작부: this.aiBalance = this.aiBalance ?? someInitialValue;
```

```
// processAiTurn() 레이즈 분기 수정안
let extra = Math.max(this.roundBaseBet, Math.floor(playerBet * raiseMultiplier / this.roundBaseBet) * this.roundBaseBet);
let aiTotal = playerBet + extra;

// AI가 감당 가능한 만큼으로 clamp
if (aiTotal > this.aiBalance) {
    aiTotal = this.aiBalance;    // AI 올인 처리
    extra = Math.max(0, aiTotal - playerBet);
}
this.aiBalance -= aiTotal;
this.potMoney += aiTotal;
```

이렇게 하면 AI도 "감당할 수 있는 만큼만" 베팅하게 되어, 무제한 판돈 증식 버그가 근본적으로 차단된다.

5.2 [4.3 대응] 폴드 임계값을 절대 배수 대신 팻 성장률 기준으로 재설계

현재 `playerBet >= roundBaseBet * 2` 조건 대신, "이번 베팅이 라운드 시작 시점 팻 대비 얼마나 커졌는가"를 기준으로 삼는다.

```
// 기존
if (Math.random() < foldBias && playerBet >= this.roundBaseBet * 2) { ... }

// 수정안: 하프/쿼터도 폴드를 유도할 수 있도록 임계값을 낮추거나,
// 애초에 ante를 낮춰 하프 금액 자체를 키우는 방향도 고려 가능
const foldThreshold = this.roundBaseBet * 1.2; // 예시: 임계값 완화
if (Math.random() < foldBias && playerBet >= foldThreshold) { ... }
```

혹은 반대로 "콜/하프/쿼터는 절대 폴드를 유도하지 못한다"를 의도된 설계로 유지하려면, 버튼 설명에 "하프/쿼터는 탐색용, 폴드를 유도하려면 올인이 필요합니다" 같은 안내 문구를 추가하는 것도 대안이다.

5.3 [4.4 대응] foldBias 구간을 실제 점수 분포에 맞게 재조정

```
// 기존 (0.08~0.18 구간은 도달 불가능한 사문화된 조건)
let foldBias = normalizedStrength < 0.08 ? 0.52 : normalizedStrength < 0.18 ? 0.26 : 0.06;

// 수정안: 실제 점수 분포(곳 100~110 / 중간축보 700~750 / 땡 801~810 / 광땡 990~1000)에 맞춰
// 중간축보와 땡·광땡의 폴드 확률에도 차등을 두어 원래 의도였던 3단계 반응을 복원
let foldBias;
if (normalizedStrength < 0.08) foldBias = 0.52;    // 곳
else if (normalizedStrength < 0.80) foldBias = 0.22; // 중간축보 · 땡
else foldBias = 0.05;                             // 광땡
```

5.4 [4.5 대응] 재레이즈 최대 횟수 제한 추가

```
// Game 객체에 카운터 추가
this.reraiseCount = 0;

// handlePlayerBet()의 재레이즈 분기에서
if (actualExtra > 0) {
    this.reraiseCount++;
    if (this.reraiseCount >= 3) {
        // 강제로 콜 처리 후 쇼다운으로 전환
        this.userBalance -= 0; // 이미 total 반영됨
        setTimeout(() => this.showdown(), 650);
        return;
    }
    setTimeout(() => this.processAiTurn(actualExtra, true), 500);
}
// startNewRound()에서 this.reraiseCount = 0; 으로 초기화
```

5.5 [4.2, 4.6 대응] 회계 정합성 보정

다이 시 소멸하는 팻머니를 "하우스 수익" 항목으로 누적 집계하거나(통계/업적 기능과 연계 가능), 무승부 시 `Math.floor(potMoney/2)` 로 버려지는 1원은 `Math.ceil` 과 `Math.floor` 를 번갈아 사용하거나 플레이어 쪽에 올림 처리하여 소멸 금액을 없앤다.

6. 한계 및 유의사항

- 변동성이 매우 큼:** 추천 전략의 판당 표준편차(17,986원)는 평균(1,282원)의 14배에 달한다. 본 보고서의 기대수익은 "충분히 많은 판"을 전제로 한 장기 기대값이며, 단기적으로는 손실 구간이 길게 이어질 수 있다.
- 추천 전략은 4.1의 버그에 상당 부분 의존:** 파트 D의 극단적 성장(1,000판 후 억대)은 AI 잔고 미검증 버그가 없었다면 훨씬 낮았을 것이다. 5.1의 패치가 적용되면 실제 수익률은 본 보고서 수치보다 크게 낮아질 것으로 예상된다.
- AI 스타일 분포 고정 가정:** tight/balanced/aggressive/bluffer 4종이 매판 균등 확률(25%)로 배정되는 원본 코드 기준이며, 이 분포나 각 스타일의 보정값이 바뀌면 결과도 달라진다.
- 본 시뮬레이션은 게임의 **수학적 로직만** 재현한 것으로, 실제 UI 렌더링·애니메이션·사운드는 결과에 영향을 주지 않는다.

본 보고서는 제공된 3D_셋다.html 소스코드를 기반으로 한 시뮬레이션 분석 결과이며, 코드가 수정될 경우 결과가 달라질 수 있습니다.